

EXERCISE 4

AES

FINITE FIELD

1. In this problem, we look at addition in extension fields. Compute

$$A(x) + B(x) \mod P(x)$$

in $\text{GF}(2^4)$ using the irreducible polynomial $P(x) = x^4 + x + 1$. What would happen if we change the reduction polynomial?

(1) $A(x) = x^2 + 1, B(x) = x^3 + x^2 + 1$.

(2) $A(x) = x^2 + 1, B(x) = x + 1$.

2. Compute in $\text{GF}(2^8)$:

$$\frac{x^4 + x + 1}{x^7 + x^6 + x^3 + x^2}$$

where the irreducible polynomial is the one used by AES, namely $P(x) = x^8 + x^4 + x^3 + x + 1$. Note that Table 1 contains a list of all multiplicative inverses for this field.

3. We consider the finite field $\text{GF}(2^4)$ with $P(x) = x^4 + x + 1$ being the irreducible polynomial. Find the inverses of $A(x) = x$ and $B(x) = x^2 + x$. You can find the inverses either by trial and error, i.e., brute-force search, or by applying the Euclidean algorithm for polynomials. (However, the Euclidean algorithm is only sketched briefly in this chapter.) Verify your answer by multiplying the inverses you determined by A and B , respectively.

4. Find all irreducible polynomials

(1) of degree 3 over $\text{GF}(2)$,

(2) of degree 4 over $\text{GF}(2)$.

The best approach for doing this is to consider all polynomials of lower degree and check whether you can split them into factors (other than 1 and the polynomial itself). Note that we only consider monic irreducible polynomials, i.e., polynomials with the highest coefficient equal to one.

PROGRAMMING IN PYTHON

The Galois library in Python provides an easy and efficient way to work with finite fields (Galois fields). It supports arithmetic operations like addition, multiplication, division, and finding multiplicative inverses within Galois fields.

```
1 $ pip install galois
```

To verify that ‘galois’ is installed correctly, you can run the following test script:

Date: December 11, 2024.

```

1 import galois
2
3 # Define a binary extension field GF(2^8) using the irreducible
  polynomial x^8 + x^4 + x^3 + x + 1
4 GF2_8 = galois.GF(2**8, irreducible_poly="x^8 + x^4 + x^3 + x + 1")
5
6 # Define elements in the field
7 a = GF2_8(0x57) # Element in hexadecimal (AES style)
8 b = GF2_8(0x83)
9
10 # Perform operations
11 print(f"a + b = {a + b}") # Addition in GF(2^8)
12 print(f"a * b = {a * b}") # Multiplication in GF(2^8)
13 print(f"a^3 = {a**3}") # Exponentiation in GF(2^8)
14 print(f"a / b = {a / b}") # Division in GF(2^8)
15 print(f"1 / b = {b**-1}") # Multiplicative inverse of b in
  GF(2^8)

```

Here is how you can use the galois library in Python to work with polynomials directly in a finite field like $GF(2^8)$, which is often used in AES. The galois library has built-in support for representing polynomials and performing operations on them in the finite field.

```

1 import galois
2
3 # Define the binary extension field GF(2^8) using the irreducible
  polynomial x^8 + x^4 + x^3 + x + 1
4 GF2_8 = galois.GF(2**8, irreducible_poly="x^8 + x^4 + x^3 + x + 1")
5
6 # Define the elements as polynomials in GF(2^8)
7 # Example: a = 0x57 -> polynomial: x^6 + x^4 + x^2 + x + 1
8 a = GF2_8.Poly([1, 1, 0, 1, 0, 1, 0, 1]) # Corresponds to 0x57 or
  binary: 0b1010111
9
10 # Example: b = 0x83 -> polynomial: x^7 + x + 1
11 b = GF2_8.Poly([1, 0, 0, 0, 0, 0, 1, 1]) # Corresponds to 0x83 or
  binary: 0b1000011
12
13 # Perform operations using polynomials
14 # Addition of polynomials (XOR operation in GF(2^8))
15 add_result = a + b
16
17 # Multiplication of polynomials modulo the irreducible polynomial
18 mul_result = a * b
19
20 # Exponentiation of polynomial a to the power of 3
21 exp_result = a**3
22
23 # Division of polynomial a by polynomial b
24 div_result = a / b
25
26 # Output the results in polynomial form
27 print(f"a + b = {add_result}")
28 print(f"a * b = {mul_result}")
29 print(f"a^3 = {exp_result}")
30 print(f"a / b = {div_result}")

```

INTERNAL STRUCTURE OF AES

5. We consider the first part of the ByteSub operation, i.e., the Galois field inversion. The following table provides the multiplicative inverse in $GF(2^8)$ for bytes xy used within the AES S-box, with the irreducible polynomial $P(x) = x^8 + x^4 + x^3 + x + 1$:

- (1) Using Table 1, what is the inverse of the bytes 29, F3, and 01, where each byte is given in hexadecimal notation?
- (2) Verify your answer by performing a $GF(2^8)$ multiplication with your answer and the input byte. Note that you have to represent each byte first as a polynomial in $GF(2^8)$. The MSB of each byte represents the x^7 coefficient.

6. Your task is to compute the S-box, i.e., the ByteSub, values for the input bytes 29, F3, and 01, where each byte is given in hexadecimal notation.

- (1) First, look up the inverses using Table 1 to obtain values B' . Now, perform the affine mapping by computing the matrix-vector multiplication and addition.
- (2) Verify your result using the S-box in Table 2.
- (3) What is the value of $S(0)$?

7. We consider AES with a 128-bit key. What is the output of the first round of AES if the input of the first Byte Substitution Layer consists of 128 ones, and the first subkey (i.e., k_1) also consists of 128 ones? It is best if you write the output state as a 4×4 array, as shown in Section 4.4.

8. In the following, we check the diffusion property of AES after a single round. Let $X = (X_0, X_1, X_2, X_3) = (0x01000000, 0x00000000, 0x00000000, 0x00000000)$ be the four input words (32 bits each) to AES with a 128-bit key. Note that all input bits except one have the value zero. The subkeys for the first round are denoted by $W[0], \dots, W[7]$ with 32 bits each, and are given by:

$$\begin{aligned} W[0] &= 0x2B7E1516, & W[2] &= 0xABF71588, \\ W[1] &= 0x28AED2A6, & W[3] &= 0x09CF4F3C, \\ W[4] &= 0xA0FAFE17, & W[6] &= 0x23A33939, \\ W[5] &= 0x88542CB1, & W[7] &= 0x2A6C7605. \end{aligned}$$

Perform the following computations. You can do them manually or write a computer program or use an existing one.

- (1) Compute the output of the first round of AES for the given input X and subkeys $W[0], \dots, W[7]$. Also provide all intermediate steps, i.e., the outputs after the layers SubBytes, ShiftRows, and MixColumns. It is recommended that you provide all results in a 4×4 state array.
- (2) Compute the output of the first round of AES for the case that all input bits are zero.
- (3) How many output bits have changed? Note that we only consider a single round—after every further round, more output bits will be affected. This behavior is referred to as the avalanche effect.

9. We are in the 9th round of an AES-128 decryption. Your task is to compute the final round and the corresponding plaintext. The AES-128 key is given by:

$$k_0 = (0x00112233, 0x44556677, 0x88990011, 0x22334455)$$

$$= \begin{bmatrix} 00 & 44 & 88 & 22 \\ 11 & 55 & 99 & 33 \\ 22 & 66 & 00 & 44 \\ 33 & 77 & 11 & 55 \end{bmatrix}$$

The state after the inverse byte substitution in Round 9 is

$$\begin{bmatrix} 77 & e9 & 45 & 58 \\ c4 & cf & fa & c3 \\ b8 & 66 & 87 & 95 \\ 05 & 23 & 31 & 20 \end{bmatrix}$$

For your answer, write the output state also as a 4×4 array.

10. For the following, we assume AES with a 192-bit key. Furthermore, we assume we have access to special-purpose integrated circuits that can check 3×10^7 AES keys per second.

- (1) If we use 100,000 such ICs in parallel, how long does an average key search take? Compare this period of time with the age of the universe (approximately 10^{10} years).
- (2) Assuming Moore's law will still be valid for the next few years, how many years do we have to wait until we can build a key-search machine to perform an average key search of AES-192 in 24 hours? Again, assume that we use 100,000 ICs in parallel.

11. The MixColumn operation has a somewhat surprising property: If the four input bytes all have the same value, the four output bytes are also all the same, and additionally have the same value as the input byte. (We note that this property does not lead to a security weakness of AES.)

- (1) Show that the property holds.
- (2) Compute the probability that four random input bytes take the same value.
- (3) Argue why this situation (i.e., all four input bytes of the MixColumn operation are identical) occurs infrequently in practice if AES is used with a random key.

12. Show that the two constant matrices for the MixColumn and Inv MixColumn are each other's inverses.

13. In this problem, we look at computations in the MixColumn layer on the bit level. As we saw in this chapter, the MixColumn transformation consists of a matrix-vector multiplication in the finite field $\text{GF}(2^8)$ with the irreducible polynomial $P(x) = x^8 + x^4 + x^3 + x + 1$.

Let $b = (b_7x^7 + \dots + b_0)$ be one of the (four) input bytes to the vector-matrix multiplication. Within the matrix multiplication, the byte b is multiplied with the constants 01, 02, and 03. We are interested in the bit-level equations for computing those three constant multiplications. We denote the result by $d = (d_7x^7 + \dots + d_0)$.

Your task is to express the bits d_i in terms of the input bits b_i .

- (1) Equations for computing the 8 bits of $d = 01 \cdot b$.
- (2) Equations for computing the 8 bits of $d = 02 \cdot b$.
- (3) Equations for computing the 8 bits of $d = 03 \cdot b$.

Note that in the AES specifications, "01" represents the polynomial 1, "02" represents the polynomial x , and "03" represents $x + 1$.

14. We now look at the gate (or bit) complexity of the MixColumn function, using the results from problem 4.17. We recall from the discussion of stream ciphers that a 2-input XOR gate performs a $\text{GF}(2)$ addition.

- (1) How many 2-input XOR gates are required to perform one constant multiplication by 01, 02, and 03, respectively, in $\text{GF}(2^8)$?
- (2) What is the overall gate complexity of one matrix–vector multiplication? (The gate complexity is important when implementing AES in hardware.)
- (3) What is the overall gate complexity of a hardware implementation of the entire Diffusion layer? Note that permutations require no gates.

15. Derive the bit representation for the following round constants within the key schedule:

- (1) $\text{RC}[8]$
- (2) $\text{RC}[9]$

X	Y															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	01	8D	F6	CB	52	7B	D1	E8	4F	29	C0	B0	E1	E5	C7
1	74	B4	AA	4B	99	2B	60	5F	58	3F	FD	CC	FF	40	EE	B2
2	3A	6E	5A	F1	55	4D	A8	C9	C1	0A	98	15	30	44	A2	C2
3	2C	45	92	6C	F3	39	66	42	F2	35	20	6F	77	BB	59	19
4	1D	FE	37	67	2D	31	F5	69	A7	64	AB	13	54	25	E9	09
5	ED	5C	05	CA	4C	24	87	BF	18	3E	22	F0	51	EC	61	17
6	16	5E	AF	D3	49	A6	36	43	F4	47	91	DF	33	93	21	3B
7	79	B7	97	85	10	B5	BA	3C	B6	70	D0	06	A1	FA	81	82
8	83	7E	7F	80	96	73	BE	56	9B	9E	95	D9	F7	02	B9	A4
9	DE	6A	32	6D	D8	8A	84	72	2A	14	9F	88	F9	DC	89	9A
A	FB	7C	2E	C3	8F	B8	65	48	26	C8	12	4A	CE	E7	D2	62
B	0C	E0	1F	EF	11	75	78	71	A5	8E	76	3D	BD	BC	86	57
C	0B	28	2F	A3	DA	D4	E4	0F	A9	27	53	04	1B	FC	AC	E6
D	7A	07	AE	63	C5	DB	E2	EA	94	8B	C4	D5	9D	F8	90	6B
E	B1	0D	D6	EB	C6	0E	CF	AD	08	4E	D7	E3	5D	50	1E	B3
F	5B	23	38	34	68	46	03	8C	DD	9C	7D	A0	CD	1A	41	1C

Table 1: Multiplicative Inverses in $GF(2^8)$ for AES S-box

X	Y															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Table 2: AES S-box Substitution Values in Hexadecimal
Notation for Input Byte (x, y)

Multiple-Choice Questions on AES.

1. Which of the following is the key size for AES?
 - A) 56 bits
 - B) 64 bits
 - C) 128, 192, or 256 bits
 - D) 512 bits
2. How many rounds are used in AES-128?
 - A) 8 rounds
 - B) 10 rounds
 - C) 12 rounds
 - D) 14 rounds
3. Which of the following is not a component of the AES encryption process?
 - A) SubBytes
 - B) ShiftRows
 - C) Permutation
 - D) MixColumns
4. What is the block size used in AES?
 - A) 64 bits
 - B) 128 bits
 - C) 192 bits
 - D) 256 bits
5. In AES, what is the purpose of the ShiftRows step?
 - A) To add the round key to the state
 - B) To create confusion by substituting bytes
 - C) To provide diffusion by shifting rows
 - D) To mix the columns for better security
6. In AES, what is the function of the MixColumns step?
 - A) To substitute bytes
 - B) To XOR the key with the state
 - C) To mix data within each column to provide diffusion
 - D) To shift rows for better security
7. AES is based on which kind of cipher structure?
 - A) Feistel Network
 - B) Substitution-Permutation Network (SPN)
 - C) Stream Cipher
 - D) Linear Feedback Shift Register
8. Which of the following is true about AES?
 - A) It is vulnerable to differential cryptanalysis.
 - B) It has a fixed number of rounds regardless of key size.
 - C) It uses key whitening.
 - D) It uses different numbers of rounds based on key size.
9. Which step in AES ensures non-linearity and confusion?
 - A) SubBytes
 - B) MixColumns
 - C) AddRoundKey
 - D) ShiftRows
10. What is the main purpose of the AddRoundKey step in AES?
 - A) To provide diffusion by mixing bytes
 - B) To add confusion by substituting bytes
 - C) To XOR the state with the round key
 - D) To shift rows to ensure non-linearity
11. In AES, which operation is primarily responsible for achieving diffusion?
 - A) SubBytes
 - B) ShiftRows
 - C) MixColumns

- D) AddRoundKey
12. How does AES achieve key expansion?
- A) Through a single XOR operation
 - B) Using a key schedule that generates round keys
 - C) By repeating the main key multiple times
 - D) Through permutation and substitution only